TEAM WORKSHOP

Docker

From *“works on my machine”* to shipping the machine.

slides + live demos

AGENDA

On this call

1 · Fundamentals ~15 min

Why containers exist, what a container actually is — vs VMs, vs real servers — and how images relate to containers.

2 · Building images ~20 min

What an image really is — layers, tarballs — then Dockerfiles, caching, multi-stage, whiteouts, dive.

3 · Docker run in practice ~15 min

What `docker run` really does, then ports, env vars, volumes, networks — wiring a real two-container stack by hand.

4 · Debugging & ops ~20 min

The toolbox for when it breaks — and it will break (on purpose). Q&A.

Docker Compose — the tool that tames what section 3 builds by hand — got its own follow-up workshop.

SECTION 1

Fundamentals

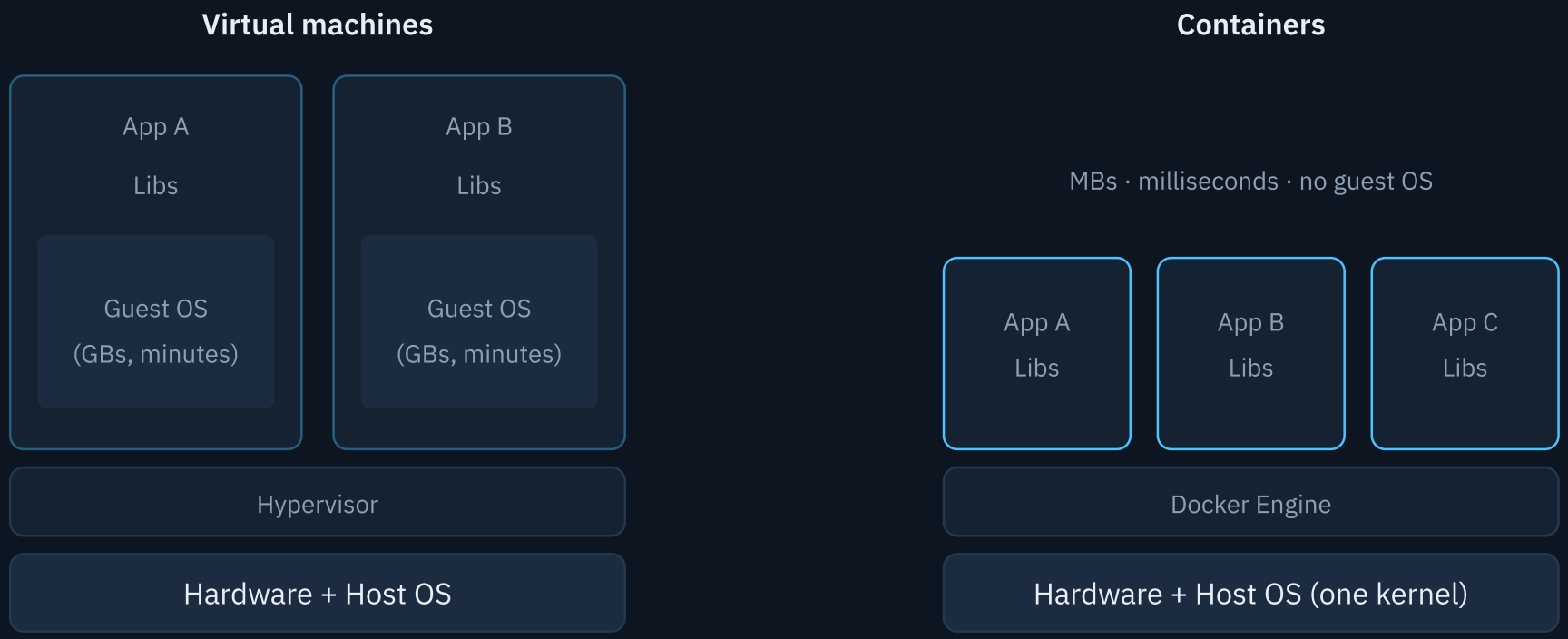
Why containers exist, what a container actually is — vs VMs, vs real servers — and how images relate to containers.

“Works on my machine”

- Your app doesn’t run alone — it needs a **Python version**, system libraries, env vars, a database, an OS with particular behavior.
- Every machine drifts: laptops ≠ CI ≠ production. Onboarding an intern takes a day of “install these 12 things”.
- Virtualenvs solve *one layer* (Python packages). Nothing pins the rest.

> THE IDEA
>
> Stop shipping code and a README of hopes. Ship the **entire runtime environment** as one artifact that runs identically everywhere.

Containers vs virtual machines



A VM virtualizes **hardware**; a container virtualizes the **operating system**. That’s why containers start in milliseconds and you can run dozens on a laptop.

A container is just a process

- Not a mini-VM. It’s a **normal Linux process** on your machine — you can see it in `ps` on the host.
- The kernel wraps it in **namespaces**: its own filesystem view, network stack, process tree, hostname.
- And meters it with **cgroups**: limits on CPU and memory.
- Isolation is the illusion; the kernel is shared.

> CONSEQUENCES YOU’LL FEEL
>
> Container stops when its main process (PID 1) exits · no Linux kernel on macOS/Windows → Docker Desktop runs a hidden Linux VM · a container is *disposable by design*.

A container is not a little server

	REAL MACHINE / VM	CONTAINER
kernel	its own	borrowed from the host
boot	firmware → kernel → init (systemd) → dozens of services	none — just `exec` your process as PID 1
processes	sshd, cron, log daemons, your app	your app. That's it. (ideally)
get in	ssh	`docker exec`
logs	/var/log, journald	stdout/stderr → `docker logs`
disk	persistent by default	ephemeral by default; volumes opt in

DEEP NUGGET · PID 1

Your process runs as PID 1, a role normally played by init — so default signal handling is off (no default SIGTERM action) and nobody reaps zombies. If `docker stop` hangs 10s then kills: that's this. Fixes: `--init` flag, or an entrypoint that handles signals (uvicorn does).

Images vs containers

Image

- Read-only **template**: filesystem + metadata (env, default command)
- Built once, shared via a registry (Docker Hub, ECR, …)
- Like a **class** — or a frozen meal

Container

- A **running instance** of an image + a thin writable layer
- Start, stop, delete — cheap and disposable
- Like an **object** — the meal, heated and half-eaten

One image → many containers. Delete a container, the image is untouched. Anything written inside a container dies with it — that's a feature, and volumes (the Compose workshop) are the answer.

LIVE DEMO 1

First contact

```
$ docker run -it --rm python:3.12-slim python
# a Python REPL, on a machine that doesn't have this Python installed
$ docker run -d -p 8000:80 --name web nginx
$ docker ps                    # the container is a process with a name
$ curl localhost:8000          # nginx, from inside the container
$ docker stop web
$ docker ps                    # empty!
$ docker ps -a                 # …but the corpse is here — Exited (0)
$ docker rm web
```

Watch for: pull-once-then-instant starts · `ps` vs `ps -a` · nothing installed on the host.

SECTION 2

Building images

What an image really is — layers, tarballs — then Dockerfiles, caching, multi-stage, whiteouts, dive.

The Dockerfile

```
FROM python:3.12-slim   # start from a base image
WORKDIR /code           # cd (and mkdir -p)
COPY --from=ghcr.io/astral-sh/uv:0.7.19 /uv /bin/uv
COPY pyproject.toml uv.lock ./  # host → image
RUN uv sync --frozen    # install exactly the lockfile
COPY app/ app/
CMD ["uvicorn", "app.main:app", …]
```

- A build script where **every instruction creates a layer**
- `RUN` executes *at build time*; `CMD` is the default command *at run time* — the classic mix-up
- Built with `docker build -t name:tag .` — and that `.` is the **build context**, which deserves its own slide →

The build context

The `.` in `docker build .` is not “where the Dockerfile lives” — it’s **which files the builder is allowed to see**.

```
$ docker build --progress=plain -t notes:v2 .
#              raw daemon logs instead of the collapsing TTY view
=> [internal] load build context
=> => transferring context: 2.1kB     # ← your folder, packed up and SENT
```

- `docker build` never reads your disk directly: the client **tars the context directory and ships it** to the builder — that’s the “transferring context” line in every build
- `COPY` can only reach files **inside** the context: `COPY ../secrets.env` fails by design — nothing outside the folder you named can end up in an image
- What ships is what your instructions **can reach**: BuildKit transfers only the paths your `COPY`s reference — but the moment a Dockerfile says `COPY . .`, that’s the *entire* context, stray `data/` and `.git` included (proof in a live demo later this section)
- `.dockerignore` trims what gets shipped — that’s why it speeds up builds and stops junk from busting the cache (it gets its own slide later this section)
- The Dockerfile is just read from the context root by default — `-f path/to/Dockerfile` picks another one, the context stays whatever you passed

> SMELL TEST
>
> A build that hangs on “transferring context” for seconds — or a context measured in MB for a small app — means the context is dragging junk along. Check `.dockerignore` first.

Base images — and their slim versions

`FROM` picks your starting filesystem — and it's the single biggest lever on image size, before any Dockerfile cleverness. The same Python, three wrappings (sizes measured on this machine):

TAG	WHAT'S INSIDE	SIZE
`python:3.12`	full Debian: compilers, headers, git — the kitchen sink, "just in case"	1.62 GB
`python:3.12-slim`	the **same Debian**, stripped to what running Python needs	205 MB
`python:3.12-alpine`	different OS: Alpine + musl libc — tiny, with caveats	79.5 MB

- The full image exists for convenience: things compile without thought. You pay ~1.4 GB for tools your app never runs
- **slim is the team default for Python**: same distro, same glibc, every wheel works — just smaller
- alpine uses **musl**, not glibc: some Python wheels don't ship musl builds and compile from source (slow) or misbehave — measure before adopting
- Pin the distro release too when reproducibility matters: `python:3.12-slim-bookworm`

LIVE DEMO 2 · BASE IMAGES

v1 vs v2: the base image gap

```
$ grep ^FROM v1.Dockerfile v2.Dockerfile
v1.Dockerfile:FROM python:3.12
v2.Dockerfile:FROM python:3.12-slim
$ docker build --progress=plain -f v1.Dockerfile -t notes:v1 .
#              plain = full daemon logs, step by step — nothing collapses
$ docker build --progress=plain -f v2.Dockerfile -t notes:v2 .
$ docker images notes            # 1.72 GB vs 319 MB — the SAME app
$ docker images python           # the bases themselves: 1.62 GB vs 205 MB
```

Watch for: ~1.4 GB of difference explained almost entirely by the `FROM` line. v2 still has v1's naive layer order — fixing *that* is the next few slides.

Layer cache: order is everything

v2 — cache buster

```
COPY . .
RUN uv sync
```

Any code edit changes the `COPY` layer → cache invalidated **from that line down** → every dependency reinstalls. Every. Single. Build.

v2.1 — cache friendly

```
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen
COPY app/ app/
```

Code edits only invalidate the last `COPY`. Dependencies reinstall **only when the lockfile changes**. Rebuilds drop from minutes to ~1s.

Rule: order instructions from **least → most frequently changing**.

LIVE DEMO 2 · CACHE RACE

The cache race

```
# v2.1 = v2 with ONE change: the lockfile is copied and synced BEFORE the code
$ grep -E '^(COPY|RUN)' v2.Dockerfile v2.1.Dockerfile
$ docker build --progress=plain -f v2.1.Dockerfile -t notes:v2.1 .    # warm it once

# touch the code, rebuild both — same base, same flags, only the ORDER differs:
$ sed -i '' 's/Workshop Notes API/Workshop Notes API v2/' app/main.py
$ time docker build --progress=plain -f v2.Dockerfile -t notes:v2 .     # re-syncs everything
$ time docker build --progress=plain -f v2.1.Dockerfile -t notes:v2.1 . # CACHED — ~0.5s
$ sed -i '' 's/Notes API v2/Notes API/' app/main.py   # undo the edit
```

Watch for: `CACHED` lines in the v2.1 output · near-identical images (319 vs 318 MB), ~6× rebuild gap even on this tiny app — and it grows with every dependency you add.

ENTRYPOINT vs CMD

Both say what runs at start — docker **concatenates them**: `ENTRYPOINT + CMD` = the command line.

```
ENTRYPOINT ["uv"]                # the fixed part — what this container IS
CMD ["run", "uvicorn", "app.main:app"]   # default arguments — easily replaced

$ docker run notes               # runs: uv run uvicorn app.main:app
$ docker run notes --version     # runs: uv --version   ← your args REPLACED CMD
$ docker run --entrypoint sh -it notes   # overriding the fixed part takes a flag
```

	ENTRYPOINT	CMD
role	the executable — “what it is”	default arguments — “how, by default”
overridden by	`--entrypoint` flag	anything typed after the image name
alone	acts as the whole command	acts as the whole command

- App images usually need only `CMD` (ours do); `ENTRYPOINT` shines for **CLI-tool images** where the binary is fixed and users pass arguments
- Official images often combine both: postgres runs an `ENTRYPOINT` init script whose last act is `exec "$CMD"`

> ALWAYS USE THE EXEC FORM
>
> `CMD ["uvicorn", …]` (JSON array), never `CMD uvicorn …` — the shell form wraps your app in `/bin/sh -c`, so **sh becomes PID 1**, signals never reach your app, and every `docker stop` hangs 10 s then kills. Same PID 1 story as section 1.

.dockerignore — the forgotten file

```
.git
.venv
__pycache__
*.pyc
.idea
```

- `COPY . .` copies *everything* in the context: your `.git` history, your 500 MB virtualenv, editor junk
- Bloats the image, slows the context upload, **busts the cache** on files that don't matter
- Can leak secrets (`.env`, keys) into image layers — layers are *forever*, even if a later layer deletes the file

LIVE DEMO 2 · DOCKERIGNORE

120 MB you never asked to ship

```
# a teammate adds a local dataset — gitignored, so nobody sees it in review
$ mkdir -p data && dd if=/dev/urandom of=data/dataset.bin bs=1m count=120
$ du -sh data/                                  # 120M

$ docker build --progress=plain -f v1.Dockerfile -t notes:v1 .
# => transferring context: 125.87MB          ← COPY . . drags it in, every build
$ docker images notes                         # v1: 1.72 GB → 1.98 GB. oops.

$ docker build --progress=plain -f v2.1.Dockerfile -t notes:v2.1 .
# => transferring context: 265B              ← v2.1's targeted COPYs never reach data/

$ echo "data/" >> .dockerignore               # the real fix — protects EVERY Dockerfile
$ docker build --progress=plain -f v1.Dockerfile -t notes:v1 .   # context tiny, image back to 1.72 GB
```

Watch for: **gitignored ≠ dockerignored** — invisible to review, shipped to the daemon · v2.1's targeted COPYs dodge it by luck; `.dockerignore` fixes it by policy, for v1 and v2 too.

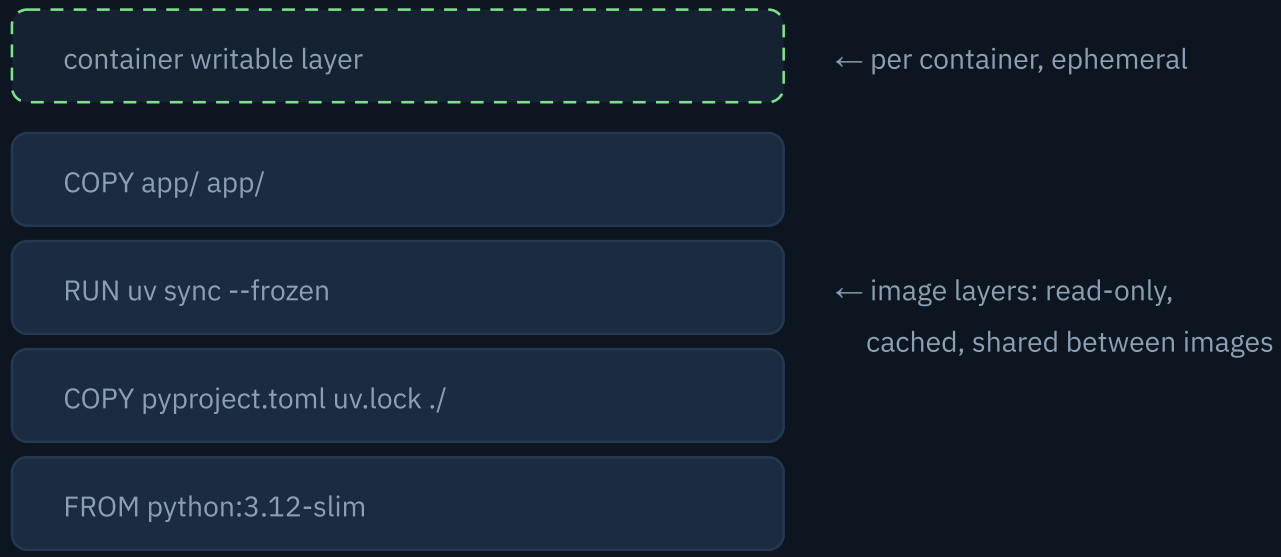
Deleting a file doesn’t shrink the image

Layers are **append-only diffs**. A later `RUN rm` doesn’t remove bytes — it adds a *whiteout marker* (`.wh.big.bin`) that hides the file at runtime. The bytes still ship in the earlier layer. Proof, from this repo:

```
FROM alpine:3.20                                  #   ~8 MB
RUN dd if=/dev/zero of=/big.bin bs=1M count=100  # +105 MB layer
RUN rm /big.bin                                   # +4.1 kB — the whiteout!
# ls inside: no big.bin anywhere. docker images:
notes:whiteout   119MB
```

- Fix 1: **clean up in the same layer** — `apt-get install … && rm -rf /var/lib/apt/lists/*` as *one* RUN
- Fix 2: don’t create the junk — `uv sync --no-cache` (our v1 quietly ships 49 MB of uv cache)
- Fix 3: **multi-stage** — the nuclear option: a fresh image simply never sees the junk
- Same physics is why a `RUN rm secret.key` is a security incident, not a cleanup

Images are stacks of layers



Each Dockerfile instruction = one layer. Unchanged instruction (and everything above it in this stack) = reused from cache.

Layers are content-addressed and **shared**: ten images on `python:3.12-slim` store that base *once*. This is also why instruction **order** decides your build speed — exactly what the cache race demonstrated.

An image is just tarballs + JSON

No magic format. `docker save` an image and untar it:

```
$ docker save notes:v3 | tar -t
blobs/sha256/1cf6d7cb8acf…   # config JSON
blobs/sha256/297b376ee6a7…   # layer (tar)
blobs/sha256/2b48c98adf89…   # layer (tar)
blobs/sha256/5464b19ed1ee…   # manifest: the parts list
…
index.json                   # entry point → manifest digest(s)
manifest.json                # legacy docker-load equivalent
oci-layout                   # {"imageLayoutVersion":"1.0.0"}
```

- **oci-layout**: a one-line version stamp — “this folder follows the **OCI (Open Container Initiative) Image Layout** spec”, the vendor-neutral standard docker, podman, containerd and registries all speak
- **index.json**: the entry point — points (by digest) to the manifest blob(s); one index can carry several images or CPU architectures (that’s how multi-arch tags work)
- **Manifest**: the parts list — config + layer digests in order (`manifest.json` is its legacy `docker load` twin)
- **Config JSON**: env, default CMD, build history
- **Each layer**: a plain tar of the *filesystem diff* that instruction made
- Blob names are **sha256 of content** → identical layers dedupe everywhere: disk, registry, network — that’s the “Already exists” lines during a pull

Reading `docker history`

The image's recipe, one row per layer — newest on top, base image at the bottom:

```
$ docker history notes:v2.1
IMAGE          CREATED BY                                      SIZE
c32fa975eb99   CMD ["uvicorn" "app.main:app" …]                0B        # metadata only
<missing>      COPY app/ app/                                  24.6kB    # ← our code
<missing>      ENV PATH=/code/.venv/bin:…                      0B
<missing>      RUN uv sync --frozen --no-dev …                 48.6MB  # ← our deps
<missing>      COPY pyproject.toml uv.lock ./                  156kB
<missing>      WORKDIR /code                                   8.19kB
<missing>      COPY /uv /bin/uv                                35.9MB    # ← uv itself
<missing>      CMD ["python3"]                                 0B        # ┐
<missing>      RUN … apt-get install … python3.12 …            44.6MB    # ├ python:3.12-slim's
<missing>      ENV PYTHON_VERSION=3.12.14                      0B        # ┘ own history
```

- Read **bottom-up** — that's the order the instructions ran, and the base image's own history is included (there is no hard boundary: your image *is* the base plus your rows)
- **SIZE is what that layer added.** `0B` rows (`ENV`, `CMD`, `EXPOSE`) touch only the config JSON — they're free
- The big rows are your optimization targets — here 48.6 MB of deps (plus 35.9 MB of uv itself) vs 24.6 kB of code, which is exactly why deps install *before* code in the cache ordering
- `<missing>` is cosmetic: intermediate layers just aren't separate tagged images on your machine — nothing is broken
- `--no-trunc` shows the full commands when the `…` hides the interesting part

The bottom of the stack: `scratch`

Every `FROM` chain ends at `scratch`: a reserved name for the **empty image** — zero layers, zero bytes, no shell, no `ls`, no libc. It isn't even pullable; it's a keyword meaning "start from nothing".

```
# how official base images are born —
# debian's actual Dockerfile is just:
FROM scratch
ADD rootfs.tar.xz /    # a root filesystem
CMD ["bash"]           # built outside docker

# alpine: the same trick with
# alpine-minirootfs-*.tar.gz
```

```
# using it directly — ship ONE static
# binary (replay: demo/scratch/):
FROM scratch
COPY busybox /busybox
ENTRYPOINT ["/busybox","echo","hi"]

$ docker run --rm scratch-test
hello from scratch
$ docker images scratch-test   # 2.2MB
```

- Works only for **static binaries** (Go, Rust, musl builds): there's no libc to link against — a dynamically linked binary dies at start with a confusing `no such file or directory` (it means the *linker* is missing, not your binary)
- No shell also means no `docker exec` into it — debugging happens from the outside
- The practical middle ground: **distroless** images — libc, CA certs, timezones included, still no shell or package manager
- Turtles all the way down: every base image on Docker Hub bottoms out at a rootfs tarball sitting on `scratch`

LIVE DEMO 2 · LAYERS

Layer anatomy, live

```
$ docker history notes:v2.1       # uv sync 48.6 MB · uv itself 35.9 MB · code 24.6 kB

# whiteouts: delete a file, image doesn't shrink
$ docker build --progress=plain -f whiteout.Dockerfile -t notes:whiteout .
$ docker images notes:whiteout  # 119 MB — alpine is ~8 MB and big.bin was DELETED
$ docker history notes:whiteout # dd keeps its 105 MB; the rm is a 4.1 kB whiteout

$ docker save notes:v3 | tar -t | head   # an image is tarballs + JSON, no magic
```

Watch for: reading history bottom-up · the 4.1 kB `rm` layer next to the 105 MB it failed to remove.

dive — an x-ray for images

```
$ brew install dive
$ dive notes:v1
# left: layers + their sizes
# right: the file tree, with this
#   layer's additions highlighted
# Tab to switch panes, Ctrl+U to
#   show only changed files
$ CI=true dive --ci notes:v1
# headless mode — fail CI builds
# that waste too many bytes
```

- Answers "**which instruction added these 200 MB?**" — click a layer, see exactly which files it wrote
- Flags **wasted bytes**: files duplicated or deleted across layers (whiteouts show up instantly)
- Honest caveat: its "efficiency %" only counts *cross-layer* waste — v1 scores 99% while being 1.7 GB. The score is not the point; **browsing where bytes live is** (go find `/root/.cache` in v1)

LIVE DEMO 2 · OPTIONAL

X-ray and scratch

```
$ dive notes:v1                # Tab: file tree · Ctrl+U: this layer's files
# find /root/.cache in the uv-sync layer — 49 MB shipped for nothing
$ CI=true dive --ci notes:v1   # headless mode — the CI guardrail

# FROM scratch, live (files in demo/scratch/)
$ cd scratch
$ C=$(docker create busybox:musl) && docker cp $C:/bin/busybox . && docker rm $C
$ docker build --progress=plain -f scratch.Dockerfile -t scratch-test .
$ docker run --rm scratch-test  # hello from scratch — a 2.2 MB image
```

Stretch material — run it when ahead of schedule. Encore: repeat with plain `busybox` (dynamically linked) and watch scratch refuse it.

Multi-stage builds

```
FROM python:3.12-slim AS builder   # stage 1: heavy lifting
COPY --from=ghcr.io/astral-sh/uv:0.7.19 /uv /bin/uv
WORKDIR /code
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev           # → /code/.venv

FROM python:3.12-slim              # stage 2: fresh, clean image
WORKDIR /code
COPY --from=builder /code/.venv .venv  # the deps — without uv itself
COPY app/ app/
```

- Build tools, installers, caches stay in the builder stage — **only the result ships**: here the final image has the `.venv` but no uv binary at all (verified: v2.1 318 MB → v3 265 MB)
- Dramatic when there's heavy tooling to leave behind (Go: ~800 MB toolchain → ~10 MB image; C extensions, node_modules builds)
- Same idea powers "build frontend with node, serve with nginx" images

Multi-stage, level 2

One real-world Dockerfile: shared deps, tests that never ship, and a frontend built by a toolchain the final image never sees:

```
FROM python:3.12-slim AS deps          # ① the expensive part, done once
COPY --from=ghcr.io/astral-sh/uv:0.7.19 /uv /bin/uv
WORKDIR /code
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev          # prod deps only → .venv

FROM deps AS test                      # ② builds ON TOP of deps
RUN uv sync --frozen                   # adds the dev group: pytest, ruff…
COPY app/ tests/ ./
RUN .venv/bin/pytest                   # red tests = failed build. never ships

FROM node:22-slim AS frontend          # ③ a different toolchain entirely
COPY ui/ .
RUN npm ci && npm run build            # → dist/

FROM python:3.12-slim                  # ④ runtime: only the results ship
WORKDIR /code
COPY --from=deps /code/.venv .venv     # prod deps — no uv, no dev group
COPY --from=frontend dist/ static/
COPY app/ app/
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0"]
```

```
$ docker build --target test .   # CI: stop at ②, test, ship nothing
$ docker build -t app .              # prod: ② is skipped, no node inside
```

- Stages can start **from earlier stages** — `test` reuses `deps`, so dependencies install once
- **Dev dependencies never reach prod**: the `dev` group from pyproject.toml installs only inside the `test` stage (plain `uv sync` includes it; `--no-dev` skips it), and runtime copies `.venv` from `deps` — not from `test` — so pytest/ruff add zero bytes to the shipped image
- `--target` picks where to stop: **one Dockerfile drives both CI and prod**
- BuildKit **skips stages the target doesn't need** and builds independent ones in parallel — the test stage costs prod builds nothing
- `COPY --from=` also accepts any *image* — that's exactly how uv gets in above, and the same trick borrows single files from anywhere: `COPY --from=nginx:1.27 /etc/nginx/nginx.conf ./`

LIVE DEMO 2 · MULTI-STAGE

Multi-stage, measured

```
$ docker build --progress=plain -f v3.Dockerfile -t notes:v3 .
$ docker images notes            # 1.7 GB → 318 MB → 265 MB — v3 left uv behind

$ docker run --rm -d -p 8000:8000 --name api notes:v3 && curl localhost:8000
$ docker stop api
```

Watch for: v3 smaller than v2 with zero effort at runtime — the uv binary and its bootstrap stayed in the builder stage.

Tags vs digests

Two ways to name the same image — one for humans, one for machines:

- A **tag** is a **movable pointer** to an image: `notes:v2`, `python:3.12-slim` — repointable at any time, exactly like a git branch
- The **digest** is the commit hash: `sha256:` of the image's manifest — immutable, verifiable, never reused. The tag can point somewhere else tomorrow; the digest can't

```
$ docker images --digests python:3.12-slim
python  3.12-slim  sha256:7a8b475003c4…          # the tag's target TODAY

FROM python:3.12-slim@sha256:7a8b475003c4…       # pin by digest: this build
                                                 # can never silently change
```

Tags are mutable — the `:latest` lie

- `:latest` is just the *default tag name*. It is **not** "the newest version" and it doesn't auto-update
- Your machine happily keeps a months-old `:latest` until you `docker pull`
- Every tag has this problem, not just `:latest`: `python:3.12-slim` quietly repoints when the base is rebuilt with security patches — same name, different bytes
- Whether that's a feature (free security fixes) or a risk (unreviewed change) is a *choice* — make it consciously, per environment

> TEAM RULE OF THUMB
>
> Pin versions everywhere reproducibility matters: `FROM python:3.12-slim` not `FROM python` (digest-pin when supply-chain rigor matters), a committed `uv.lock` (and `uv sync --frozen` so builds fail rather than drift), unique tags (git SHA) for deployed images.

One image name, many CPUs

An image is built **for an architecture** — and your laptop's may not be your server's:

- Apple Silicon builds **linux/arm64** by default; most servers and CI runners are **linux/amd64**. Ship the wrong one and the container dies with `exec format error` — "works on my machine", hardware edition
- `docker build --platform linux/amd64` cross-builds via emulation (Rosetta/QEMU) — correct output, noticeably slower builds
- Real multi-arch uses buildx: `docker buildx build --platform linux/amd64,linux/arm64 -t img --push` — one tag, one manifest **index** pointing at per-arch manifests (the same `index.json` from the tarballs slide!)
- That's why `docker pull python:3.12-slim` just works on both your Mac and CI — the registry serves each machine its own architecture automatically

> TEAM RULE OF THUMB
>
> Build for prod's platform in CI (native amd64 runners beat emulation), and treat the platform-mismatch WARNING on `docker run` as a deploy bug waiting to happen — not noise.

LIVE DEMO 2 · PLATFORMS

Cross-building, live

```
$ docker image inspect notes:v2.1 --format '{{.Os}}/{{.Architecture}}'
linux/arm64                          # built on this M-series Mac → arm64

$ docker build --progress=plain --platform linux/amd64 -f v2.1.Dockerfile -t notes:amd64 .
$ docker image inspect notes:amd64 --format '{{.Os}}/{{.Architecture}}'
linux/amd64                          # same Dockerfile, different CPU target

$ docker run --rm notes:amd64 python -c "import platform; print(platform.machine())"
WARNING: image's platform (linux/amd64) does not match host (linux/arm64)
x86_64                               # running x86 code on ARM — emulated, slower
```

Watch for: the cross-build pulling a *different* set of base layers · the run WARNING — on a real deploy that line means someone shipped the wrong architecture.

Image checklist

- ✓ slim/alpine base, version-pinned
- ✓ dependencies installed *before* code is copied
- ✓ `.dockerignore` exists and covers .git / venv / caches
- ✓ multi-stage when there’s a build step
- ✓ non-root `USER` in the final stage
- ✓ one process per container
- ✓ config via env vars, never baked in
- ✗ secrets in any layer, ever

SECTION 3

Docker run in practice

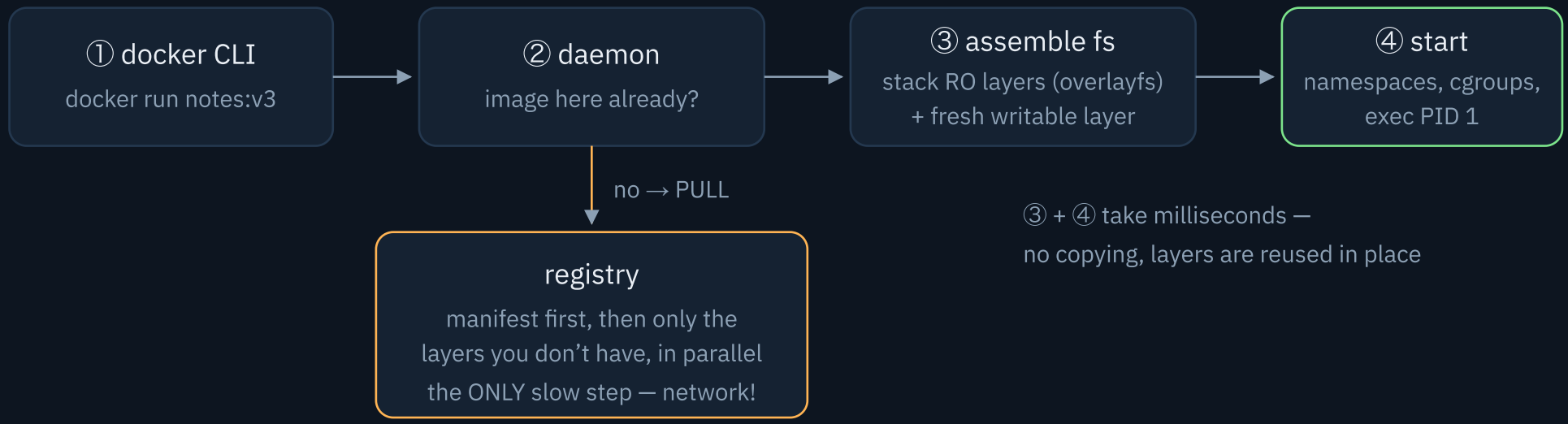
Ports, env vars, volumes, networks — everything a real container needs, one flag at a time.

Anatomy of `docker run`

```
docker run -it --rm -d -p 8000:8000 -e KEY=val --name api image:tag  # cmd
```

FLAG	MEANING
`-it`	interactive terminal — for shells and REPLs
`--rm`	delete the container when it exits (keep your machine clean)
`-d`	detached — run in the background
`-p 8000:8000`	publish port — **host:container**, in that order
`-e KEY=val`	set an environment variable (the 12-factor config channel)
`--name api`	a human name instead of `eloquent_hopper`

What `docker run` actually does



Every fresh machine pays the pull: CI runners, autoscaled nodes, new teammates' laptops, every deploy.

A 1.7 GB image vs a 290 MB image is the difference the whole team feels daily — that was section 2.

Ports: publishing to the host

```
docker run -p 8000:8000 notes:v3
#           host  :container — host first, always
```

- Containers get their own network stack — their ports are **invisible** until published
- `-p 9000:8000` → app listens on 8000 inside, you hit `localhost:9000` outside. Two apps can both use “their” 8000
- App must bind `0.0.0.0`, not `127.0.0.1` — inside the container, localhost means *the container*
- `-p 127.0.0.1:5432:5432` publishes **to your machine only** — without the prefix, the port is open to your whole network. Databases in dev stacks deserve the prefix

Env vars: config without rebuilds

```
$ docker run -e DEBUG=1 -e DATABASE_URL=postgresql://… notes:v2.1
$ docker run --env-file .env notes:v2.1     # a file of KEY=val lines
$ docker exec api env                      # what did it ACTUALLY get?
```

- Env is **the** config channel: one image, different env → dev, staging, prod. Never bake config into the image
- Set at `docker run` time — changing a value means recreating the container, not rebuilding the image
- Secrets ride env at runtime, **never** `COPY`d into layers (layers are forever — section 2)
- Debug rule: what the app *sees* is what `docker exec <c> env` prints — not what your shell exports

Volumes: data that survives

```
$ docker run -d --name db -v pgdata:/var/lib/postgresql/data postgres:16-alpine
#                           named volume — created on first use, owned by docker
$ docker run -v ./app:/code/app notes:v2.1      # bind mount — your real folder, mirrored in
$ docker volume ls
$ docker volume inspect pgdata
```

- The container's writable layer **dies with the container** — volumes are the opt-out
- **Named volume** (`pgdata:`): docker-managed storage — for *state*: databases, uploads. Survives `docker rm`
- **Bind mount** (`./app:`): a host folder mirrored inside — for *code*: edit in your editor, the container sees it instantly
- Rule of thumb: **code in bind mounts, state in volumes**

Networks: containers talk by name

```
$ docker network create demo
$ docker run -d --name db  --network demo postgres:16-alpine
$ docker run -d --name api --network demo \
    -e DATABASE_URL=postgresql://workshop:workshop@db:5432/notes notes:v2.1
```

- Containers on the same user-defined **network** reach each other **by container name** — Docker runs an internal DNS
- No published ports needed for container↔container traffic: `db:5432` works with nothing exposed to the host

> CLASSIC MISTAKE
>
> `localhost:5432` inside the api container is the *api container itself*, not your host, not the db. If code says localhost and it works on the host but not in Docker — this is why.

LIVE DEMO 3 · A REAL STACK

The stack, by hand

```
$ docker network create demo
$ docker run -d --name db --network demo -v pgdata:/var/lib/postgresql/data \
    -e POSTGRES_USER=workshop -e POSTGRES_PASSWORD=workshop -e POSTGRES_DB=notes \
    postgres:16-alpine
$ docker run -d --name api --network demo -p 8000:8000 \
    -e DATABASE_URL=postgresql://workshop:workshop@db:5432/notes notes:v2.1
$ curl -X POST "localhost:8000/notes?text=by-hand" && curl localhost:8000/notes
$ curl localhost:8000                    # "storage": "postgres" — one env var did that

# the volume outlives the container:
$ docker rm -f db                        # kill the database entirely
$ docker run -d --name db --network demo -v pgdata:/var/lib/postgresql/data \
    -e POSTGRES_USER=workshop -e POSTGRES_PASSWORD=workshop -e POSTGRES_DB=notes \
    postgres:16-alpine
$ curl localhost:8000/notes              # notes still there — pgdata survived
```

Leave this stack running — section 4 is going to break it.

This doesn’t scale

That demo took four commands and a paragraph of flags — and it’s a *tiny* stack:

```
docker network create demo
docker run -d --name db --network demo -e POSTGRES_USER=… -e POSTGRES_PASSWORD=… \
  -v pgdata:/var/lib/postgresql/data postgres:16-alpine
docker run -d --name api --network demo -p 8000:8000 -e DATABASE_URL=… \
  -v ./app:/code/app notes:v2.1 uvicorn … --reload
```

- Every flag lives in someone’s shell history — nothing is versioned, reviewed, or shared
- Startup order, restart behavior, cleanup: all manual, all forgettable
- The fix is **Docker Compose**: the whole stack in one committed file, `up` / `down` as single commands — that’s the follow-up workshop

SECTION 4

Debugging & ops

The toolbox for when it breaks — and it will break (on purpose).

The debugging toolbox

COMMAND	QUESTION IT ANSWERS
`docker ps -a`	what's running — and what *died* (exit codes!)
`docker logs -f --tail 100 api`	what did it say before it died
`docker exec -it api bash`	let me look around inside (sh if bash is missing)
`docker inspect api`	full config: env, mounts, network, IP — everything
`docker stats`	live CPU/memory — who's eating the laptop

Debugging order: `ps -a` → `logs` → `exec` → `inspect`. It resolves 95% of container issues.

The deeper toolbox

COMMAND	WHEN YOU NEED IT
`docker run -it --entrypoint sh IMAGE`	container dies instantly? skip its CMD and look around the image yourself
`docker logs --since 10m --timestamps api`	just the recent window, with times — not 3 days of scrollback
`docker cp api:/code/app/main.py .`	pull any file out — works on **stopped** containers too
`docker diff api`	every file the container changed vs its image — spot rogue writes
`docker top api`	the processes inside, without exec'ing in
`docker inspect -f '{{.State.ExitCode}}' api`	one field, scriptable — Go templates beat piping to grep
`docker events --since 10m`	the daemon's own feed: OOM kills, restarts, dies — things logs never show

The first row is the underrated one: when a container won’t even start, debug the *image* instead of the corpse.

Failure bestiary

port is already allocated

Something owns that host port. `docker ps` for a forgotten container, `lsof -i :8000` for a host process. Or just map another port.

Exited (1) immediately

App crashed on boot. `docker logs <id>` — the answer is always there. Often: bad env var, missing file, wrong CMD.

Exited (137) / OOMKilled

137 = SIGKILL, usually the kernel enforcing a memory limit. Check `docker inspect` → OOMKilled, raise the limit or fix the leak.

connection refused to db

Wrong host (`localhost` instead of the other container's name), db not accepting connections yet, or the containers sit on different networks.

Limits & disk space

Resource limits

```
$ docker run --memory 512m --cpus 1.5 …
$ docker update --memory 512m api   # adjust a RUNNING
$ docker stats                      # verify what's en
```

Unlimited by default — one leaky container can starve the machine. Limits also make prod behavior reproducible locally (Compose and orchestrators expose the same knobs).

“Docker ate my disk”

```
$ docker system df    # who's guilty
$ docker system prune  # stopped containers,
                       # dangling images, caches
$ docker system prune -a --volumes
  # ⚠ nukes unused images AND volumes
```

Old images and build cache accumulate for months. Prune periodically; read what `-a --volumes` will delete before saying yes.

LIVE DEMO 4

Break it, then find it

```
# sabotage the stack from demo 3 (openly — "I'm the intern now"):
$ docker rm -f api && docker run -d --name api --network demo -p 8000:8000 \
    -e DATABASE_URL=postgresql://workshop:WRONG@db:5432/notes notes:v2.1
$ curl localhost:8000/health  # 503 — database unreachable
$ docker ps                   # both "Up". the picture lies
$ docker logs api             # ← "password authentication failed". there it is
$ docker exec -it api sh      # env | grep DATABASE — the value it REALLY got
$ docker inspect api          # …or the same truth from outside
# fix: recreate api with the right password → /health is ok again

# bonus round: occupy the port first
$ python3 -m http.server 8000 &
$ docker run -p 8000:8000 …   # "port is already allocated"
$ kill %1
# cleanup: docker rm -f api db && docker network rm demo
```

Same playbook every time: **status** → **logs** → **exec** → **inspect**.

RECAP

What you take home

1 · Fundamentals

A container is a **process with isolation**, not a small server — it borrows the host kernel, runs your app as PID 1, and is disposable by design. The only slow part of `docker run` is the pull.

2 · Building images

An image is **cached layers** (just tarballs + JSON). Order instructions **least → most changing**, sync deps from the lockfile before copying code, multi-stage so tooling never ships. `dive` when in doubt.

3 · Docker run in practice

Four flags carry a real stack: `-p` publishes, `-e` configures (never bake config in), `-v` persists — **code in bind mounts, state in volumes** — and `--network` lets containers find each other **by name**.

4 · Debugging

One playbook, always in this order: **`ps`** → **`logs`** → **`exec`** → **`inspect`**. Exit 137 = out of memory, "port allocated" = something else owns it, and `docker system prune` when disk fills.

IF YOU REMEMBER ONE THING

Ship the environment, not install instructions — and when it breaks, the answer is already in `docker logs`.

Everything from today — slides, demo app, all Dockerfiles, cheat sheet — is in the workshop repo: `git clone` and replay it. Next up: **Docker Compose**, where the by-hand stack becomes one committed file.

FIN

Questions?

Then go containerize something.